

Concept of Operations (ConOps) — Claude apps gateway (AWS GovCloud)

Companion to the security-review package ([architecture.md](#) , [network-access-controls.md](#) , [security-review-2026-07.md](#)) for the RMF/FedRAMP ATO submission. Where those documents describe *what the system is built from*, this one describes *how it is operated and used*: who the users are, what they do, how the system behaves in normal and degraded modes, and what the operating organization must provide. It is written for ISSO/AO reviewers who have not read the repository.

This deployment is a **client-configurable template**, not a single fielded system: every organization-specific value (gateway FQDN, Okta issuer, email domains, model IDs, network CIDRs, account IDs) is a CloudFormation parameter or a `scripts/deploy.env` variable. Placeholders in this document (`claude-gateway.example.com` , `grafana-admins`) stand in for those values.

Where the architecture is already documented, this ConOps **references** the diagrams and inventories rather than restating them. When code and prose disagree, the CloudFormation templates in `cloudformation/` are authoritative.

1. Purpose and scope

1.1 Purpose

The system is a **self-hosted gateway that lets developers use Claude Code (the Anthropic coding agent) entirely inside the organization's AWS GovCloud boundary**. Developer requests never leave to a public Anthropic endpoint; model inference is served from **Amazon Bedrock** within GovCloud (`us-gov-west-1`), authenticated by the organization's own **Okta** identity provider, and recorded in the organization's own audit and observability stores. The gateway exists so that an approved-model, identity-attributed, fully in-boundary path to Claude can be offered to developers on managed Windows laptops without a public internet dependency on the AI provider.

1.2 Scope — what the system is

- An **internal Application Load Balancer + ECS Fargate gateway** service that terminates developer TLS, authenticates users via Okta OIDC, and proxies inference to Bedrock (`cloudformation/02-gateway.yaml`).
- An **RDS PostgreSQL** store for session and spend state (`cloudformation/01-database.yaml`).
- An **offline Windows client rollout**: a mirrored, integrity-verified `claude` binary and a **no-admin, user-scope installer** — it writes the binary and a user-settings `env` block (telemetry tags, update lockdown, CA trust) and needs no elevation; forced gateway login, when wanted, is a separate GPO/MDM admin channel (`client/mirror-claude-release.sh` , `client/Install-ClaudeCode.ps1` ; [client-config.md](#)).

- An **optional usage/cost observability stack**: Amazon Managed Prometheus (AMP) and a self-hosted Grafana behind the same ALB and IdP (`cloudformation/03-observability.yaml`). Usage metrics reach AMP through an **ADOT collector that runs as a co-resident sidecar inside the gateway task** (`cloudformation/02-gateway.yaml`), not as a standalone service.

1.3 Scope — what the system is not

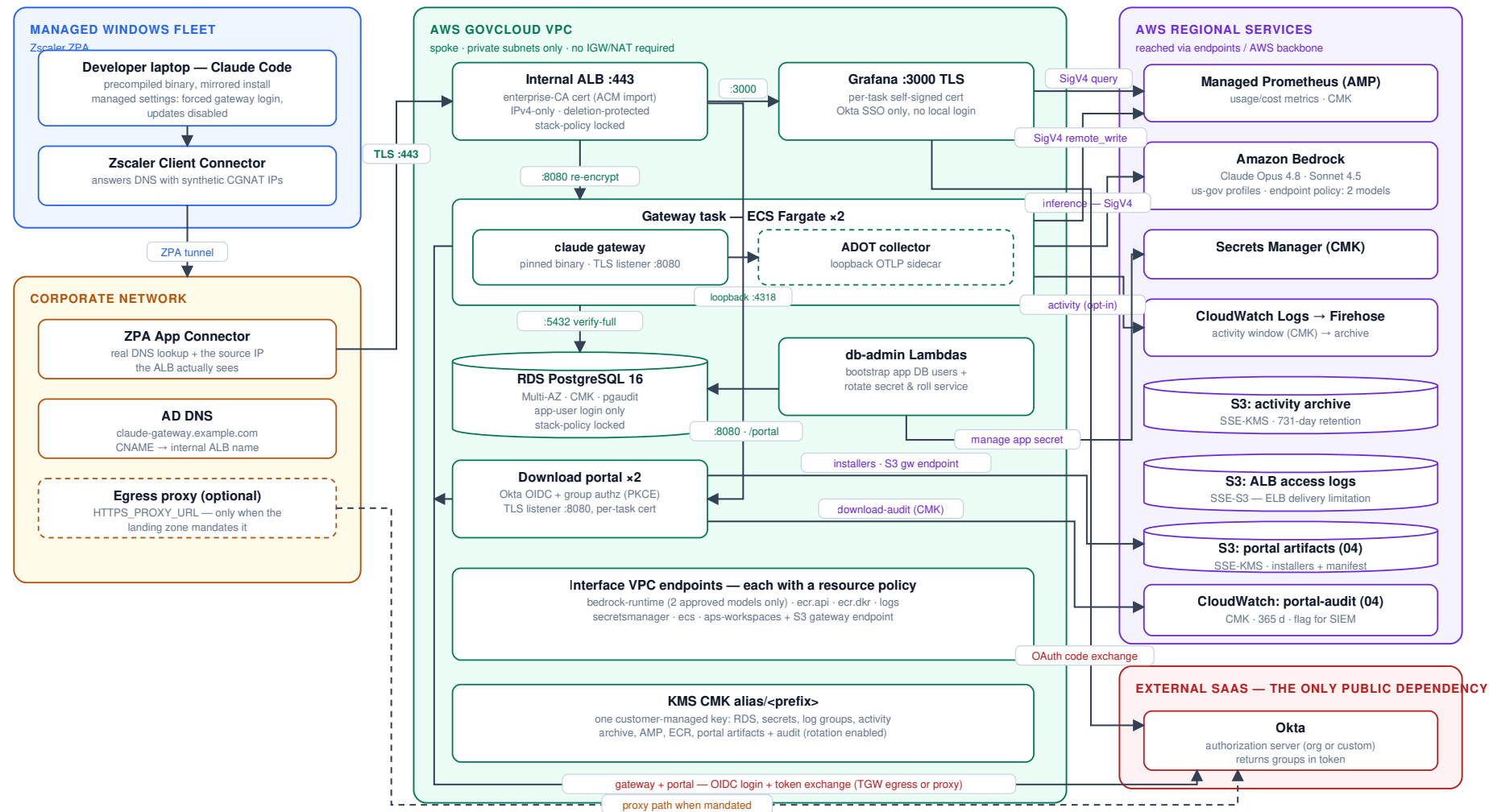
- **Not public-facing.** The ALB is internal; there is no public ingress path (`architecture.md §1`, "No public ingress").
- **Not a general AI gateway.** Exactly two Bedrock models are exposed — Opus 4.8 and Sonnet 4.5 — enforced at the application, IAM, and VPC-endpoint layers (`cloudformation/02-gateway.yaml models: block`; `architecture.md §10` note 5).
- **Not dependent on Anthropic-hosted infrastructure at runtime.** The client fleet never contacts Anthropic: binaries are mirrored and verified, and all auto-update paths are disabled by the user-settings `env` block the installer writes (`DISABLE_UPDATES / DISABLE_AUTOUPDATER`), backstopped by the mirror-only network path (`client/mirror-claude-release.sh`, `client/Install-ClaudeCode.ps1`).
- **Not a Node/npm distribution.** By decision (2026-07-15), only the precompiled native `claude` binary is fielded (`CLAUDE.md` "User decisions").

1.4 System boundary

The authorization boundary comprises four trust zones — the managed Windows fleet (Zscaler ZPA), the corporate network (App Connectors, AD DNS, egress proxy), the AWS GovCloud workload VPC (all server components), and one external SaaS dependency, Okta. Bedrock and every other AWS service are reached over private endpoints or the AWS backbone. The single-page boundary view is reproduced below; the split views and per-hop detail are in `architecture.md §1–§2`.

System architecture & trust boundaries

Claude apps gateway in AWS GovCloud us-gov-west-1 — every arrow is TLS unless flagged



Boundary facts: no public ingress (internal ALB behind ZPA); no public egress from the inference path (Bedrock endpoint, 2-model policy); clients never contact Anthropic (mirrored, verified binaries; updates disabled).

2. System overview

At a glance (full detail in `architecture.md` §1–§2):

- Developers on managed Windows laptops run `claude`, which reaches an **internal ALB** over TLS via **Zscaler ZPA**. The ALB re-encrypts to the **ECS Fargate gateway** tasks (per-task TLS), which authenticate the user against **Okta OIDC** and proxy inference to **Bedrock** over a private interface endpoint whose policy admits only the two approved models.
- **Session and spend state** lives in **RDS PostgreSQL 16**; the gateway connects as a least-privilege application role, never the RDS master user.
- **Usage telemetry** (tokens, cost, model, and stamped user identity) flows from the gateway to a **co-resident ADOT collector sidecar** (localhost, inside the gateway task), which remote-writes into **AMP**; it is visualized in a self-hosted **Grafana** that also authenticates through Okta.
- An optional **installer download portal** — a small ECS Fargate service on the same ALB (path `/portal`) — lets developers self-serve a pre-configured installer ZIP after Okta login and group check, with every download and denial written to a dedicated audit log (`cloudformation/04-download-portal.yaml`; §5.1, §5.4).
- Everything at rest is encrypted with a single **customer-managed KMS key** (the one documented exception is the ALB access-logs bucket, which uses SSE-S3 because ELB log delivery does not support KMS — `architecture.md` §9).

The deployment is three CloudFormation stacks (`01` database, `02` gateway, `03` observability) plus an optional fourth (`04` download portal), five container images, and a `deploy.env`-driven set of scripts. Stack dependencies and deploy order are in `architecture.md` §8.

3. Users and roles

The system has four operational actor classes. None of the day-to-day roles holds the RDS master credential.

3.1 End-user developers

- Run `claude` on managed, Zscaler-secured Windows laptops via a **no-admin**, entirely user-scope install to the per-user profile (`client/Install-claudeCode.ps1`; the installer refuses a SYSTEM-context run and writes no machine/policy settings — only a user-settings `env` block). Forced gateway login, when the org wants it, is delivered separately by GPO/MDM (`client-config.md`).
- Authenticate interactively through **Okta SSO** at first `/login` and receive a gateway session (`cloudformation/02-gateway.yaml` `oidc:` and `session:` blocks).
- Are gated at sign-in by **allowed email domain** (`allowed_email_domains` in `cloudformation/02-gateway.yaml`, from the `ALLOWED_EMAIL_DOMAINS` parameter). Okta **group** claims are available but are **not yet enforced** as gateway access control: the `groups` scope and the `managed.policies` group-match block are present but commented out in the template (`cloudformation/02-gateway.yaml`, lines around the `oidc:` and `managed:` sections). Reviewers should read gateway authorization today as "authenticated Okta user in an approved email domain," with per-group policy as an available, unexercised extension.

- Obtain the installer either from an operator/MDM push or **self-service from the download portal** (when stack 04 is deployed): they browse to `/portal` on the gateway FQDN, authenticate through Okta, pick their **Team** and **Cost Center** from configured dropdowns, and receive a single ZIP whose generated `install.cmd` bakes those choices in (§5.1; `cloudformation/04-download-portal.yaml`; `docker/portal/app.py`).

3.2 Platform operators

- Deploy and maintain the system with the `deploy.env`-driven scripts in `scripts/` (fixed order: `cert` → 01 → build four images → 02 → DNS/Zscaler → verify → 03 → Grafana secret → re-run 02; `docs/test-run-runbook.md`).
- Operate from a host with Docker, AWS credentials, and egress; an **in-VPC admin/build host** is admitted to the interface-endpoint security group when `ADMIN_CLIENT_SG_ID` is set, so its AWS API calls are not black-holed by the endpoints' private DNS (`network-access-controls.md` §1; `security-review` batch, endpoint-SG cross-stack reachability).
- Never inject the RDS master secret into any task; the deploy tooling writes secrets via mode-600 `file://` temp files, never on the command line (`.claude/rules/security.md`; `scripts/common.sh` `put_secret_and_roll`).

3.3 Security and audit consumers

- Read the **DB audit trail** (`pgaudit` → CloudWatch), **ALB access logs** (S3), the **installer download-audit log** (when the portal is deployed — see §5.4), and, when enabled, the **AI activity stream** (see §5.4).
- The activity stream is treated as highly sensitive (bash commands, tool inputs, file paths per user): it is **opt-in**, IAM-only, CMK-encrypted, and flagged for SIEM subscription (`.claude/rules/security.md`; `cloudformation/03-observability.yaml`; `FORWARD_ACTIVITY_LOGS` in `scripts/deploy.env.example`).

3.4 Break-glass database administrator

- The RDS master role (`gw`) is **break-glass only** — held by humans for emergencies and by the `db-admin` Lambda, never by a running gateway task (`architecture.md` §4; `.claude/rules/security.md`). Its RDS-managed 7-day rotation therefore affects no running workload.

3.5 Grafana / dashboard role model

Grafana authenticates against the **same Okta issuer** as the gateway, with its own client and redirect URI (`/grafana/login/generic_oauth`) and **strict Okta-group → role mapping**: membership in `grafana-admins` (the `GRAFANA_ADMIN_GROUP` parameter, default `grafana-admins`) maps to Grafana Admin, and a user in no mapped group is **denied** (`architecture.md` §3; `cloudformation/03-observability.yaml`; `docs/okta-request-email.md`). The local login form is disabled; the bootstrap `admin` account is break-glass only, reachable only by redeploying with `GRAFANA_DISABLE_LOGIN_FORM=false`.

3.6 Download-portal access group

The portal (optional stack 04) authorizes on **Okta group membership**: the `AccessGroup` parameter (`ACCESS_GROUP` in `deploy.env`, placeholder default `claude-gateway-users`) names **one or more groups** (comma-separated), and a member of **any** listed group may download the installer; everyone else is denied — and the denial is audited (§5.4). The parameter is named generically so the **same group(s) can gate the gateway** once its commented-out group policy (§3.1) is enabled, giving one org-managed membership set for both surfaces (`cloudformation/04-download-portal.yaml`; `scripts/deploy.env.example`). Unlike the gateway and Grafana, the portal's group check is enforced today, which makes the **Okta groups-claim configuration an operating prerequisite** for the portal (§8.2).

4. Operational environment

The system is fielded into an **AWS Landing Zone** with the following characteristics (CLAUDE.md "Landing zone"; `security-review-2026-07.md` target profile):

- **Hub-and-spoke with Transit Gateway** (not VPC peering), **central egress**; the workload VPC is a **no-NAT spoke** in the target profile. Because there is no local NAT, the no-NAT path relies on interface/gateway VPC endpoints for AWS APIs (`CREATE_SUPPORTING_ENDPOINTS`, `CREATE_BEDROCK_ENDPOINT`, `CREATE_AMP_ENDPOINT` in `scripts/deploy.env.example`).
- **Client access via Zscaler ZPA**: developer laptops reach the internal ALB through a ZPA application segment for the gateway FQDN (TCP 443), served by App Connectors that route to the workload VPC over the Transit Gateway. TLS inspection must **not** be enabled on that segment — the client pins the gateway certificate fingerprint (`docs/networking-request-email.md` §3).
- **Server-side egress prerequisite**: the gateway and Grafana containers dial the **Okta issuer** for OIDC discovery and token exchange. This traffic is server-originated (no Zscaler user identity), so the workload VPC's central egress path needs an explicit **ALLOW + SSL-inspection exemption** for the Okta issuer FQDN (`docs/networking-request-email.md`, "Server-side egress"). This is the single open org prerequisite blocking the first end-to-end run (CLAUDE.md Status).
- **Corporate DNS**: a single CNAME in the corporate zone points the gateway FQDN at the ALB's `internal-*.elb.amazonaws.com` name — a public record returning private IPs, so no split-horizon/private-hosted-zone is required; App Connectors must be able to resolve the corporate CNAME (`docs/networking-request-email.md` §2; `security-review-2026-07.md` B1).

Per-source-IP attribution is deliberately **not** relied upon: ZPA collapses all users to a handful of App Connector IPs, so identity is carried in the Okta → gateway-session chain, not the network layer (`architecture.md` §3; `security-review-2026-07.md` B8).

5. Operational scenarios

The narrative walkthroughs below are the core of this ConOps. Each traces a real flow through the fielded system and cites the file that implements it.

5.1 Developer onboarding and first login

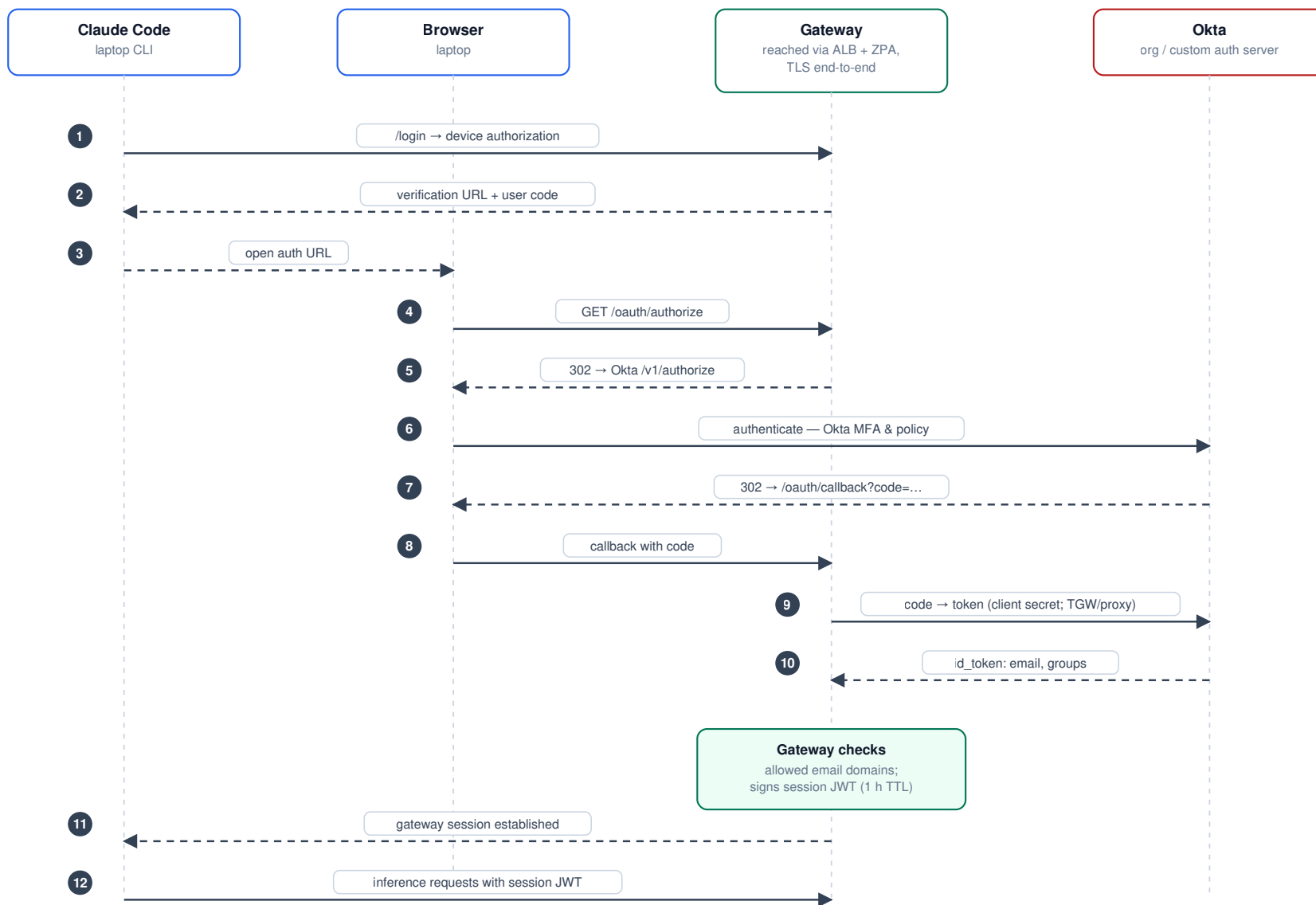
1. **Offline install.** An operator has mirrored the pinned `claude` release into an internal share; the mirror **fails closed** on integrity — it refuses to proceed without GPG verification unless `ALLOW_UNVERIFIED_MANIFEST=1` is set explicitly (`client/mirror-claude-release.sh`; `.claude/rules/security.md`). The developer obtains the installer one of two ways:
2. **Self-service (portal, when stack 04 is deployed).** The developer browses to `https://<gateway-fqdn>/portal`, signs in through Okta (authorization-code flow with PKCE, enforced `AccessGroup` membership — §3.6), selects **Team** and **Cost Center**, and downloads a single ZIP: the verified `claude.exe`, `Install-ClaudeCode.ps1`, and a generated `install.cmd` that invokes the installer with `-GatewayUrl`, `-Sha256`, `-Team`, `-CostCenter` (and update lockdown) already baked in — one double-click, no parameters to get wrong. The ZIP contents come from the portal's CMK-encrypted artifacts bucket, published from the verified mirror output by `scripts/publish-portal-release.sh` — the portal never fetches from the internet (`docker/portal/app.py`; `cloudformation/04-download-portal.yaml`).
3. **Operator/MDM push.** The developer (or an MDM push) runs `Install-ClaudeCode.ps1` directly from the internal share.

Either way, the installer places the binary in the per-user profile **without admin rights** and writes a **user-settings** `env` **block** that locks down auto-updates (`DISABLE_UPDATES=1`, `DISABLE_AUTOUPDATER=1`) and stamps telemetry tags; the developer points the CLI at the gateway interactively at first `/login` (step 2 below), and the gateway pushes central config after login via `/managed/settings`. Forced gateway-only login, when the org wants a client-side lock, is delivered by GPO/MDM, not this installer (`client/Install-ClaudeCode.ps1`; `client-config.md`).

2. **First login (Okta OIDC).** The developer runs `claude`, chooses the cloud gateway login, and is taken through the Okta authorization-code flow (PKCE, org authorization server). On success the gateway issues a session JWT that carries the Okta identity. The full sequence is diagrammed in `architecture.md` §3.

Developer authentication — Okta OIDC via the gateway

managed settings force gateway login; no local API keys exist



Audit identity chain: Okta identity → session JWT → user.id / user.email / user.groups stamped on every request and telemetry export. Grafana repeats this flow with its own client + strict group→role mapping.

1. **Certificate fingerprint trust.** Claude Code validates the ALB certificate chain, then **pins the SHA-256 fingerprint of the leaf on first connect** (trust-on-first-use, per hostname). Operators publish the expected fingerprint (printed by `import-enterprise-cert.sh`) so the developer confirms it at first login. This is exactly why TLS inspection must not sit in front of the gateway FQDN (`docs/networking-request-email.md §3`; `docs/test-run-runbook.md §1`).

5.2 Normal use — a request

Steady-state request path (`architecture.md §2`, hop table):

Laptop `claudio` → **ZPA** (TCP 443, no inspection) → **internal ALB** (TLS, FIPS policy) → **gateway task** (ALB re-encrypts to per-task TLS on 8080) → **Bedrock** (TLS + SigV4 over the interface endpoint, two approved models only). Session and spend state is read and written in **RDS** over `verify-full` TLS, with the gateway authenticating as the least-privilege application role (`gateway_app` / `gateway_app_clone`), never the master user (`architecture.md §4`). WebSearch is disabled on gateway sessions, so the inference path has no public egress (`architecture.md §1`).

5.3 Usage and cost monitoring

Each developer's user-settings `env` block can stamp **OTEL resource attributes** — `cost_center` and `team` — via `OTEL_RESOURCE_ATTRIBUTES`, set by the installer's `-CostCenter` / `-Team` parameters (`client/Install-ClaudeCode.ps1`). The gateway additionally stamps **identity** (`user.id` / `user.email` / `user.groups` from the Okta session) onto every telemetry export (`architecture.md §3, §5`). Metrics flow gateway → **co-resident ADOT collector sidecar** → **AMP** (SigV4 remote-write) → **Grafana**, where operators authenticate through Okta SSO and read the provisioned usage/cost dashboard (`cloudformation/02-gateway.yaml`, `cloudformation/03-observability.yaml`; `architecture.md §5`). Metrics carry 150-day AMP retention.

The collector is not a network service: it runs as a **sidecar container in the gateway task**, sharing that task's boundary and network namespace, and the gateway forwards to it over **loopback** (`http://localhost:4318`) — there is no over-the-wire telemetry hop. The sidecar remote-writes to AMP and to the activity log group under the **gateway task role** (scoped to this workspace and this log group only), and its OTLP listener binds `127.0.0.1` so nothing off the task can reach it. This is the resolution of finding C2: SC-8 is met by absence of network transmission (a stronger posture than an internal-CA TLS listener, and with no new PKI to custody — SC-17/SC-12), after the live run showed the gateway rejects a non-HTTPS telemetry forward URL off localhost (`security-review-2026-07.md C2` + fix log). **Pending live verification:** metrics landing in AMP through the sidecar.

5.4 Audit

Three independent audit surfaces, at increasing sensitivity (`architecture.md §5` data-flow table):

- **DB audit (pgaudit).** DDL, role changes, and writes (no bind values) are exported from RDS to a CMK-encrypted CloudWatch log group (`cloudformation/01-database.yaml`; `security-review-2026-07.md C4`).
- **ALB access logs.** Source connector IPs, URIs, and timings land in an S3 bucket (SSE-S3, IAM-only, lifecycle expiry) (`cloudformation/02-gateway.yaml`; `architecture.md §9` note).

- **Installer download audit (portal, when deployed).** Every portal download **and every denial** writes one JSON record — timestamp, verified Okta identity (`user_email`, `user_groups`), selected team and cost center, release version, binary SHA-256, ALB-attested source IP, outcome — to a **dedicated CMK-encrypted CloudWatch log group** (retention `PORTAL_AUDIT_RETENTION_DAYS`, default 365; flag for SIEM). It is deliberately separate from, and never routed into, the activity stream below (`docker/portal/app.py`; `cloudformation/04-download-portal.yaml`).
- **AI activity stream (opt-in, highly sensitive).** When `FORWARD_ACTIVITY_LOGS=true`, bash commands, tool inputs, and file paths per user are streamed to a CMK-encrypted CloudWatch window and archived to S3 (SSE-KMS), IAM-only and flagged for SIEM subscription (`cloudformation/03-observability.yaml`; `scripts/deploy.env.example`; `.claude/rules/security.md`). Prompt content is redacted. This stream's access surface is deliberately never widened.

5.5 Administration — deploy, update, and rotation

- **Deploy / update flow.** The load-bearing order is cert → 01 database → build all four images → 02 gateway → DNS/Zscaler → `verify-gateway.sh` → 03 observability → set Grafana Okta secret → re-run 02 (`docs/test-run-runbook.md`; CLAUDE.md "Deploy model"). Scripts persist their outputs back into `deploy.env` (`set_env_var` in `scripts/common.sh`), so there are no copy-paste steps. Container image tags are immutable — an image change means a new tag and a service roll (`.claude/rules/scripts.md`).
- **Automated DB-credential rotation.** The application DB secret (`<prefix>/db-app-user`) rotates on a schedule (default 90 days, `APP_SECRET_ROTATION_DAYS`) using an **alternating-user** scheme — the db-admin Lambda flips between `gateway_app` and `gateway_app_clone` and **forces the gateway service to roll** as part of `finishSecret` (`docker/db-admin/app.py`; `architecture.md` §4). The previous credential stays valid until the next rotation. The first rotation fires at stack creation as an automatic end-to-end validation (asynchronous — see §8).
- **Manual secret rotation.** Okta client secrets (gateway, Grafana, and — when deployed — the download portal) are set out-of-band via `set-okta-secret.sh` / `set-grafana-oidc-secret.sh` / `set-portal-oidc-secret.sh`, which prompt hidden, write via mode-600 `file://`, and roll the consuming service (`scripts/common.sh` `put_secret_and_roll`). The JWT session-signing secret has a documented prepend → roll → remove runbook (`architecture.md` §6).
- **Portal operations (optional stack 04).** Deployed after 02 (independent of 03): build/push the portal image (`build-and-push-portal.sh`), deploy `deploy-download-portal.sh`, set the portal's Okta secret, then publish the pinned release to the artifacts bucket with `publish-portal-release.sh` `<version>` — which reuses the GPG-verified mirror output. Serving a **new client version** to developers is that one publish command plus updating `PORTAL_RELEASE_VERSION` (`docs/om-runbooks.md` §5; `docs/test-run-runbook.md`).

6. Modes of operation

6.1 Normal

All three stacks healthy: developers log in through Okta, inference is served from Bedrock, telemetry flows to Grafana, and audit surfaces are recording.

6.2 Degraded

- **Observability stack (03) down → gateway unaffected.** Telemetry forwarding is optional and gated on the observability stack existing; the deploy-gateway guard only disables forwarding on a definitively-missing stack, and a gateway with no OTLP target continues to serve inference. Usage dashboards go stale; developer service does not (`cloudformation/02-gateway.yaml` `telemetry toggle`; `security-review-2026-07.md` self-review notes on the telemetry guard).
- **Okta unreachable → no new logins, existing sessions continue.** OIDC is the authentication front door; if the issuer is unreachable (or the server-side egress ALLOW/exemption is missing), **new** logins fail closed, but sessions already holding a valid JWT keep working until their TTL expires (`SESSION_TTL_HOURS`; `cloudformation/02-gateway.yaml` `session: block`). This is the failure the current org prerequisite would otherwise cause at boot (`CLAUDE.md` Status).
- **Collector shares the gateway task lifecycle — and fails CLOSED by default (AU-5).** The ADOT collector is a sidecar in the gateway task, not a separate service, so there is no independent collector redeploy: a gateway roll cycles the collector with it. With the default `TelemetryFailClosed=true`, the sidecar is Essential and health-checked and the gateway container waits on it: a transient collector crash restarts in place (container restart policy), but a collector that stays failed or hangs unhealthy **stops the task** — the gateway does not serve traffic while telemetry/audit processing is down; ECS replaces the task and the ALB routes to the peer task meanwhile. Orgs that prefer availability over auditability set `TELEMETRY_FAIL_CLOSED=false` (telemetry may then gap silently while inference keeps serving) and record that deviation in the SSP. On every task stop — including secret-rotation service rolls — stop ordering is gateway-first, collector-last with a 120 s drain, so the final telemetry buffer gets a best-effort flush to AMP/CloudWatch rather than being dropped on the spot (delivery is not guaranteed — a failing AMP endpoint at shutdown can still lose the last batch). The **end-to-end control is the missing-telemetry alarm** (observability stack): container health proves the collector is alive, while the alarm proves samples are actually landing in AMP — it fires after `MISSING_TELEMETRY_ALARM_MINUTES` (default 15) of ingestion silence, covering IAM/endpoint breakage, misconfiguration, and AMP-side failure that no health check can see (`security-review-2026-07.md` C2; O&M runbook 9 triage).
- **Download portal down → no self-service installs, gateway unaffected.** The portal is an optional, separate stack behind its own `/portal` listener rule; its failure (or absence) does not touch the inference path. Installers remain available from the internal share via the operator/MDM route (§5.1).

6.3 Maintenance

- **Image updates.** Rebuild and push the affected image under a **new immutable tag**, then roll the service; always push rebuilt images *before* a stack update that changes what the task definition expects (`.claude/rules/scripts.md`).
- **Certificate rotation.** The enterprise leaf is imported into ACM and does **not** auto-renew; a `DaysToExpiry` alarm fires at 30 days, and re-import re-triggers a one-time client fingerprint-trust prompt, so operators coordinate a heads-up before each renewal (`architecture.md` §6; `docs/networking-request-email.md` §1).
- **Teardown / replacement discipline.** The ALB and RDS instance are protected three ways (deletion protection, fixed names, stack policy denying replace/delete); several encryption-at-rest choices (RDS CMK, AMP CMK) are **day-one decisions** that cannot be changed by an in-place update (`architecture.md` §8; `.claude/rules/cloudformation.md`).

Detailed operations-and-maintenance procedures are collected in the O&M runbooks document, `docs/om-runbooks.md`.

7. Security posture summary and accepted risks

Stated up front, per the review-package convention. The source of truth for finding-by-finding status is `security-review-2026-07.md`; the accepted-risk register is `architecture.md §10`.

The implemented posture: a single customer-managed KMS key encrypts everything at rest (bar the ALB-logs bucket, an ELB platform limitation); TLS in transit on every network hop (the one former exception — the gateway→collector telemetry leg — is now a loopback sidecar with no network transmission, below); RDS `verify-full`; pgaudit; VPC-endpoint and IAM policies scoped to the two approved models and this account's exact ARNs; explicit (no default-allow) security-group egress; and a least-privilege application DB user with self-rolling rotation. Full batch status (A deploy-breakers, B ZPA/landing-zone prerequisites, C FedRAMP hardening C1–C11, D correctness, C12 least-privilege DB user) is in `security-review-2026-07.md`.

Accepted / deferred risks a reviewer should see immediately:

- **C2 (formerly an accepted partial risk) — WITHDRAWN, not carried.** The review had accepted the gateway→collector OTLP hop as plaintext-but-SG-scoped. The live test run **disproved** that posture — the gateway refuses a non-HTTPS telemetry forward URL unless the host is localhost, so the hop could never run — so the accepted risk was withdrawn (2026-07-22) and the collector moved to a **co-resident localhost sidecar** in the gateway task (§5.3). There is now no plaintext telemetry leg to accept: SC-8 is met by absence of network transmission, with no new PKI to manage (`security-review-2026-07.md C2 + fix log`; `architecture.md §10 note 1`). This is surfaced here rather than silently dropped because it reverses a decision recorded in earlier review packages.
 - **C9 — S3 Object Lock / WORM on audit buckets is deferred**, by decision (2026-07-15). Revisit if AU-9 WORM retention is mandated (`security-review-2026-07.md C9`; `architecture.md §10 note 3`).
 - **ALB access-logs bucket uses SSE-S3, not the CMK** — an AWS platform limitation (ELB log delivery does not support KMS), not an oversight; the bucket blocks public access and expires on a lifecycle (`architecture.md §9, §10 note 2`).
-

8. Assumptions, constraints, and dependencies

8.1 Assumptions and constraints

- **GovCloud model availability.** Only **Opus 4.8** (`us-gov.anthropic.claude-opus-4-8`) and **Sonnet 4.5** (`us-gov.anthropic.claude-sonnet-4-5-20250929-v1:0`) are available in `us-gov-west-1`; Sonnet 4.6 / Sonnet 5 are not. Model IDs must be verified against the Bedrock console before changing defaults (`scripts/deploy.env.example`; CLAUDE.md "GovCloud model availability").
- **Template, not a deployment.** No organization-specific value is hardcoded; each is a CloudFormation parameter or a `deploy.env` variable (`.claude/rules/security.md`).
- **Client distribution is the precompiled native binary only** (decision 2026-07-15); Grafana auth is Okta SSO; Object Lock is deferred (CLAUDE.md "User decisions").

8.2 Organizational dependencies (prerequisites)

These must be provided by the operating organization before the system can operate; templates for the requests exist in the repo:

- **Enterprise-CA TLS certificate** for the gateway FQDN (serverAuth EKU), imported into ACM (docs/networking-request-email.md §1; scripts/import-enterprise-cert.sh).
- **Internal DNS CNAME** for the gateway FQDN at the ALB name (docs/networking-request-email.md §2).
- **Zscaler policy**: a ZPA app segment (or ZIA bypass) for the gateway FQDN with no TLS inspection, **and** the server-side egress ALLOW + SSL-inspection exemption for the Okta issuer FQDN (docs/networking-request-email.md §3).
- **Okta OIDC application** — a confidential Web app on the org authorization server, all redirect URIs registered (gateway, Grafana, and — if the portal is deployed — /portal/oauth/callback), groups returned, and the admin group provisioned (docs/okta-request-email.md).
- **Okta groups claim configured — hard prerequisite for the portal**. The groups scope alone yields group membership in **neither** the ID token nor userinfo on an org authorization server; an Okta admin must configure a groups **claim**, or the portal denies every user. The gateway and Grafana tolerate this today (email-domain gating; Grafana needs groups too but is optional); the portal does not (docs/okta-request-email.md ; docs/om-runbooks.md).

8.3 Verification status — verified on paper vs. verified live

Per the "done means verified live" discipline (.claude/rules/process.md), the following is stated honestly for reviewers:

- **First end-to-end test run is IN PROGRESS** (as of 2026-07-17). Proven live so far: DB bootstrap and app-user auth, RDS verify-full TLS, offline image builds on a hardened host, ALB + access logs, and the endpoint-SG reachability model (CLAUDE.md Status).
- **Currently blocked** on one org prerequisite — the Zscaler ALLOW + SSL-inspection exemption for the Okta issuer FQDN on server-side egress — which the gateway needs for OIDC discovery at boot (CLAUDE.md Status; docs/networking-request-email.md).
- **Still unexercised** and therefore not yet claimed as fielded: gateway steady state and end-to-end developer login, Grafana Okta login, secret rotation, the activity archive, and the **download portal end-to-end** (live Okta round-trip with the groups claim, a real ~100 MB streamed download through the ALB, listener-rule and audit-log wiring) (CLAUDE.md Status; security-review-2026-07.md "What remains"). The system is not production-ready until the runbook's validation checklist (docs/test-run-runbook.md §9) is green.